

Selected issues in object-oriented database design

M Machura and T Jebb

In order to build robust applications that are able to perform well using object-oriented database technology, a range of factors must be considered. This paper examines some of the more important of these, such as scalability, method execution location, locking strategy and data integrity. It attempts to give general guidance to the designer, emphasising the importance of physical database design and integration with other database and distributed-object technologies.

1. Introduction

Two technologies seem to be winning the battle for next-generation database applications — object-oriented database technology (OODBT) and object/relational database technology (O/RDBT). Their increasing popularity provides evidence that database developers and users have accepted object orientation as a promising software technology.

OODB and O/RDBT have their roots in early ‘manifestos’ that attempted to define the characteristics of future advanced database systems — see Atkinson et al [1] and the UCB manifesto [2], respectively. A third manifesto [3] tried to further clarify the requirements of next-generation database management systems. In terms of current standardisation activities, the two approaches are centred on the ODMG-93 standard [4] and SQL3 [5], respectively. The main difference between these technologies lies in the originally adopted outlook on databases. OODBT emphasises its object-oriented capabilities and seamless integration with object-oriented programming languages. O/RDBT stresses its relational roots, treating object-oriented concepts as extensions to the relational model. A critical discussion of both approaches was published in *Object Magazine* [6].

This paper focuses on the development of object-oriented databases (OODBs). It does not attempt to provide a detailed discussion of the OODB design process, but rather concentrates on selected issues believed to be of particular importance to designers of OODB applications.

A brief introduction to OODBT is given first, followed by a general discussion of the OODB development process. Among specific design issues presented in the paper are object locking, the location of operation execution, object integrity maintenance, physical database design, scalability,

tool support, and integration with other database and distributed system technologies.

2. Object-oriented databases

The features, expected benefits and commercial offerings of OODBT have been presented and discussed in great detail in many publications [7–10]. The limitations of the current generation of OODB tools have also been extensively researched and are now fairly well understood [11, 12].

To a large extent, the strengths and weaknesses of OODBT lie in its origins in object technology. Object technology offers database developers new features of data type abstraction and reuse of database designs. In addition to significantly better facilities for modelling real-world entities and complex relationships between them, OODBT provides clustering of objects, navigational access to objects, creation and instantiation of new classes at run time, long-lived transactions, better (in comparison with relational databases) performance for interactive graphical applications, and support for co-operative work. Together, these features fulfil the requirements of many advanced database applications that are not served sufficiently well by traditional relational database technology. Recent trends, such as the growing importance of the ODMG-93 standard and the inclusion of object-oriented ideas in relational databases, pave the way for wider acceptance of an object-oriented model in database applications.

Perhaps the single, most frequently quoted drawback, in comparison to generally accepted relational database technology, is that OODBs are no longer designed and built independently of the actual application programs. Object-oriented database-management systems (OODBMSs) have become part of the application programming environment.

In other words, OODBMSs have lost relational simplicity and data independence. It has been suggested, however, that OODB tools build on a different type of independence, namely that of class independence. In fact, the concept of encapsulation provides a form of logical data independence, which allows changes to the implementation of an object without the need to change programs or queries that use the object.

At present, the construction of OODB applications seems to be unnecessarily time-consuming. The development process is carried out at a relatively low programming level, requires careful physical design and management of memory, and involves tedious testing. Although this approach may make sense to the developers of complex object-based data management systems, it is not acceptable to the developers of typical commercial database applications. Current OODB tools do not provide the level of support which is essential for rapid prototyping and development of commercial database applications. Another problem is that the OODBMS vendors do not seem to offer the level of technical support that is taken for granted in relational database management systems. Those deficiencies must be addressed, in order for OODBT to be able to deliver the productivity and quality gains similar to those offered by relational database technology.

3. Developing OODB applications

OODBT has attracted developers' attention because of its ability to support complex and widely varying data types and the complex structures encountered when using such types. Early application areas were mainly design and engineering systems such as CAD, CAM, CASE, and other applications such as geographical information systems. More recently, areas such as telecommunications, healthcare, computer-aided publishing, finance services, general business process management and co-operative work environments are also increasingly adopting OODBT. It is interesting to note that almost all OODBMS vendors regard network management applications, particularly in the telecommunications sector, as a strategically important application area for OODBT. Such systems include customer service management, billing systems, operational support systems, remote terminal management and event notification systems.

There are an increasing number of examples of OODBs expanding from technical markets into business applications. A typical scenario is to use an OODBMS to integrate legacy databases and applications. Another important force behind OODBs is emerging at present, i.e. electronic commerce over public networks. The electronic shopping mall technology has now reached a critical mass, and OODBT combined with distributed object technology (DOT) is expected to provide the support necessary both to

manage huge databases with multimedia content and to make it readily available to millions of customers.

3.1 Understanding OODB features

OODBT is a new technology. The available OODBMSs vary considerably in terms of their features, application areas being targeted, design and technical solutions offered, and development tools provided. OODBT expertise is not easily available, so literature search and study of vendors' reference manuals is usually the first stage of an OODB project. As organisations usually cannot afford to undertake in-depth investigations of OODBT, surveys (such as that conducted by Barry & Associates [13]) that assess and compare various OODB products are an invaluable source of information.

3.2 Identifying critical application features

This is the second major obstacle on the road to building OODB applications. In order to identify critical features of a particular application, a thorough analysis of its information and data-processing requirements is needed, as well as knowledge of OODBT features that can support them. Among typical considerations are the programming environment, data and process distribution, scale of distribution, the number of concurrent users, support for schema evolution, the use of existing data, and access to legacy databases. A study of a comprehensive list of database assessment topics such as that contained in the Barry survey [13] can draw developers' attention to the most important features of the actual application.

Three general categories of OODB applications based on the primary features demanded by applications have been identified by Manola [14].

- Applications requiring object features and specialised OODBMSs

These applications tend to depend on specialised OODBMS implementations to achieve host language integration and, most importantly, the required performance. The so-called data-shipping architectures found in many OODBMSs are of particular interest to these applications. In such architectures, data is shipped from servers to clients, so that most of the database and application processing is performed at the client. Most of the applications falling within this category are engineering and design systems, but also applications that require specialised OODBMS services, e.g. financial analysis systems, telecommunications systems, or front-ends for legacy systems in client/server architectures.

- Applications requiring only object features

These applications are less dependent on specific OODBMS implementations than those in the previous category. They require support for highly structured data, as well as different types of transactions and user-defined procedures to provide specialised behaviour for extended data types. This is an area where more direct competition between OODBT, O/RDBT and relational database technology can be expected. An important class of applications in this category is integrated systems that manage large amounts of information represented by means of different data models. OODBT offers a basis for integrating information service applications.

- Applications using an object-oriented development approach

Developers of object-oriented software who need support for object persistence regard OODBMSs as a natural extension of their programming environments. A key consideration is how seamless the programming interface is. This category is becoming more popular among object-oriented software developers, but OODBMSs seem to be unnecessarily powerful tools for such applications. More demanding applications of this type are likely to evolve in the direction of one of the first two categories.

3.3 Benchmarks

The best kind of benchmark is one that models a particular application well — it is also the most time-consuming and expensive. Unfortunately, as the available benchmarks do not cover many different uses of OODBMSs, they are rather unlikely to match a particular problem domain. Another major drawback of many benchmarks is that they are based on single-user applications, which usually leads to unrealistic comparisons.

Two benchmarks have reached some level of industry acceptance:

- the OO1 Benchmark, originally developed to measure performance of database engineering applications,
- the OO7 Benchmark from the Wisconsin University, that was aimed at ‘system’ comparisons between OODBMSs.

The Wisconsin team is currently working on a new multi-user customisable benchmark generator. There are also other on-going efforts to parameterise benchmarks or build industry-specific benchmarks [15, 16]. ODMG-93 will also impact on future comparison benchmarks by supporting common OODBMS interfaces.

Performance and scalability are typically the top evaluation criteria for data management software. An OODB benchmark should give developers an idea about expected access times, response times, throughput and storage. Ideally, it should also show how well OODBMSs scale by number of users and the nature of the data manipulation. There are many other important issues that are not covered by benchmarks, such as the programming language interface, application development tools, object modelling tools, low-level foundation classes, etc.

3.4 Logical OODB design

Object models, being data oriented rather than process oriented, suit the needs of database applications quite well. The object-oriented analysis phase corresponds to the conceptual database design phase, and can be regarded as an extension of the well-established entity-relationship modelling techniques. In fact, the majority of object-oriented analysis methods use some variant of the entity-relationship approach. In traditional database systems, the conceptual design phase is followed by the logical design phase, which maps the entity-relationship model on to a particular data model used by a DBMS. In OODB applications, this mapping is unnecessary since an object-oriented data model is used right from the start and the conceptual and logical design phases amalgamate into one object-oriented analysis phase.

There are no object-oriented methodologies that have specifically been devised for OODB applications. Most object-oriented analysis methods lack mechanisms for dealing with system-level functionality and properties of collections of objects, as well as mechanisms for supporting different user views. Another problem with the current object-oriented analysis and design methods is the lack of support for distribution and concurrency. Typically, OODB application developers use a combination of several methods. For example, Objectory [17] is used to investigate system requirements by means of use cases, and another data-oriented method, such as OMT [18], is used to model problem-domain objects. It is likely that the new object-oriented Unified Modelling Language [19] will in the future replace the current generation of methods.

3.5 Physical OODB design

The last stage in the development of database applications is physical database design during which performance factors are carefully investigated and final tuning takes place. A good physical design is the key to realising the full potential of OODBs. At present physical database design is far more challenging in OODBs than in relational databases. The current generation of OODBMSs gives application developers a lot of freedom in making, and in many cases forces them to make, physical design decisions. As OODBT matures, techniques and tools will be

provided to automate to some extent the physical database design process.

4. Locking objects

Locking mechanisms have considerable influence on database overhead and performance. OODBMSs support the two basic lock types — exclusive locks (write locks) and shared locks (read locks). Some OODBMSs also support so-called dirty locks that enable a transaction to ignore any locks that other transactions may hold and read objects irrespective of whether they are being updated by other transactions or not. Applications can either use the mechanisms provided by OODBMSs to automatically lock the requested objects, or they can explicitly request particular types of locks. In general, an OODBMS implicitly grants shared locks to referenced objects, unless the application explicitly requests exclusive locks or requests that an object not be locked at all.

Another important issue in object locking is deadlock detection. Distributed OODBMSs typically use a combination of detecting cycles in local (intranode) 'wait-for' graphs and a time-out mechanism for distributed (internode) waits. The time-out limit is commonly set at a second or less, so users rarely notice when deadlocks occur. Long-lived transactions require different concurrency control techniques (e.g. check in/check out, multiversion concurrency control).

Object contention and the effects of locking become evident only with multiple users. As a rule, the more objects that a particular transaction locks, the lower the level of concurrency that the OODB can support during that transaction. Thus applications should lock only those objects that are needed in a transaction, and choose the type of lock that meets the transaction's needs.

An important aspect of object locking is lock granularity. OODB applications can lock object clusters, single objects (object-level locking), or even individual attributes within an object. At a higher level, OODB applications can lock an entire file, a segment or a page (page-level locking). Applications typically use object- or page-level locking.

Page-level locking reduces locking overhead and can provide better performance in read-only situations, but in the case of read/write conflicts performance is actually reduced. Object-level locking, on the other hand, has a higher locking overhead, but the chances of a read/write conflict are smaller. The choice of lock granularity depends on data and operations carried out on that data. In general, page-level locking should be considered if:

- many objects on a page are used only by one person, i.e. few read/write conflicts are expected among the objects on the page.

Object-level locking is more suitable, if:

- objects are small,
- frequent read/write conflicts are expected among the objects on the page, but not necessarily on the same objects.

Object locking is an important performance issue and should be considered early on in the physical database design stage. Careful distinction should be made between development environments (with few users) and production environments (with many concurrent users). Object-locking requirements may influence the choice of an OODBMS, as some OODBMSs are better tuned for fine-grained objects than for coarse-grained objects.

5. Operation execution location and lock granularity

One of the distinguishing features of current OODBMSs is the location of operation execution. Most OODBMSs currently manage only the state of objects (attribute values and relationships), leaving the behavioural aspects of objects to the object-oriented programming languages. In other words, data is stored at the server, and operations that can be applied to data are stored in the client application workspace. Such OODBMSs are called passive, as opposed to active OODBMSs that store both the state and operations of objects in the database and make it possible to process data at the server.

The majority of current OODBMSs are passive (with the exception of GemStone, Itasca and OpenODB). The benefits of storing both objects' state and operations in an OODB are:

- improved management and administration of object data and implementations (e.g. security, integrity),
- flexibility in specifying where operations execute in the network.

According to Loomis [7], the reason that most OODBMSs are passive is to benefit from close coupling with the object-oriented programming language environments. In the future, more products may evolve to become active OODBMSs.

It is worth comparing the performance impact of operation execution in the client application workspace versus that on the server. When operations are performed in the client application workspace, the data must be moved from the server to the client. Execution of these operations in the client application workspace can be very fast,

particularly if the same operations are repeatedly executed on the same data. Performing operations on the server minimises the movement of the data, and is particularly effective if different data or large amounts of data are processed.

A discussion of the impact of operation execution location and lock granularity on the performance of different OODB applications using a number of simple scenarios can be found in Barry [20].

- Small database, single user

Locking is not an issue here. If the database is small, all data may even fit into the client cache. Executing operations on the client will provide the best performance.

- Large database, single user

Again, only operation execution location needs to be considered in this case. The database is now too large for all data to fit into the client cache, so performing the operations on the server would provide the best performance without having to move the data to the client.

- Large database, data compact on pages, multi-user

Locking issues must now be considered as well. If an OODBMS executes operations on the server, page-level locking would be likely to provide better performance due to a lower locking overhead than that of object-level locking. If operations are executed in the client application space, both page- and object-level locking may be appropriate depending on the query type.

- Large database, data widely dispersed on pages, multi-user

Assuming again that an OODBMS executes operations on the server, object-level locking is likely to give better performance. Locking individual pages that are needed by a particular operation would prohibit many other concurrent users from updating the objects locked together with their pages.

A scenario with operations executed on the client and page-level locking will give poor performance. A lot of data will have to be sent to the client. Many pages will be locked for the duration of a query (operation) execution. Subsequent execution of a similar query will cause all the data to be sent to the client again.

6. Object integrity

OODBT combines two different software development perspectives — that of database application developers and that of object-oriented software developers. Tight integration of a programming language and an OODB engine means that maintaining data integrity in OODBs differs from maintaining data integrity in relational databases.

The relational database management puts enforcement of semantics into databases, by taking data integrity logic out of applications. Relational database management systems and relational database administrators are responsible for consistent enforcement of semantic constraints. In object-oriented software, data-integrity logic is buried in applications. This is partly because operations associated with objects are an obvious place to capture operational semantics, and partly because there are no high-level declarative languages in which integrity constraints of persistent objects could naturally be expressed.

A related problem is that, because of tight integration of object-oriented programming languages and OODB engines, OODB applications need to distinguish between persistent and transient objects. In traditional relational databases, the basic operations such as queries, transactions, indexes, etc, apply only to persistent objects. As OODBMSs also do not extend their transaction roll-back mechanisms to handle transient objects, OODB applications are responsible for maintaining consistency of their transient objects.

Object integrity is enforced in OODB applications at different levels and by different components of the OODBMSs.

- Programming language enforced integrity

Object-oriented programming environments within which OODB applications are developed offer this level of support for object integrity. Two basic techniques of supporting data integrity at the programming level are strong typing and operation signatures. Strong typing and operation signatures (operation names; number, names and types of operation arguments; names and types of return values; exceptions that can be raised by operations) can be viewed as a way of imposing constraints on allowable data manipulations.

- Application enforced integrity

This level of integrity control is provided by OODB applications. Although, ideally, as much integrity control as possible should be provided by the OODBMS, some integrity constraints have to be built into the applications themselves. This approach to data integrity maintenance is currently used in complex

OODB applications such as design support environments. Typically, two kinds of measure are used in such applications. The first is to control access to objects and their attributes by preventing certain operations from being carried out on them (based, for example, on user roles or requester types). The second is to check the intended object manipulations for consistency. Such checks should ensure that modified attributes have valid values (correctness) and that all required attributes have valid values (completeness). This also includes automatic checking of data integrity in all related objects that are affected by the intended operation (change propagation). It is likely that, in the future, OODBMSs will support some of these data integrity actions.

- ODBMS enforced integrity

This is the preferred level of support because it relies on general-purpose, environment-wide mechanisms for data integrity maintenance. Some of these mechanisms are provided as part of the underlying OODBMS infrastructure and are used by all OODB applications transparently. Mechanisms of this kind offer support for unique object identifiers and name spaces, transparent reading of objects from, and writing objects to, the OODB, transparent support for replication, etc. It is likely that in the future OODBMSs will also enable developers to express integrity constraints by means of declarations or rules, which are then automatically enforced by the system when objects are manipulated.

Developers should be particularly interested in the last two methods of enforcing data integrity in OODB applications. If possible, the OODBMS-enforced integrity method should be used. OODB application developers should be aware of different mechanisms offered by OODBMSs to control data integrity. Data integrity mechanisms may differ from one OODBMS to another. They may also require different levels of human intervention to apply them effectively.

In the remainder of this section four specific aspects of maintaining data integrity are discussed. Some of them are already supported by or are being currently introduced into OODBMSs; others will be supported in the future. OODB developers should understand different ways in which they can maintain data integrity either at the OODBMS level or at the application level.

6.1 General semantic constraints

A database semantic constraint notion has been an integral part of the relational model. The three basic forms of semantic constraints are uniqueness constraints, referential integrity constraints and cardinality constraints.

198 Primary keys express a uniqueness constraint; referential

integrity constraints are expressed by means of foreign keys. Cardinality constraints may be expressed as CHECK constraints. In addition, the RDB developer using SQL can designate actions that should be taken to avoid violation of a referential integrity constraint (CASCADE, SET NULL, SET DEFAULT, NO ACTION).

ODBMSs assign location-independent object identifiers (OIDs) to all persistent objects. An OID is an application-independent unique identifier that persists with an object for its entire lifetime and is valid across storage, process and machine boundaries. OODBMSs can thus use OIDs to enforce uniqueness constraints and also as the basis for the implementation of relationships. In an OODB application using C++, OIDs of persistent objects are automatically converted to pointers that are valid in the application's address space.

However, due to the different underlying models, uniqueness and relationships in OODBs and RDBs mean rather different things. In the classic relational model, uniqueness is applied to real-world information, whereas in the object model it implies a distinct container of information. Relationships in the relational model again apply to real-world values and are used to maintain data integrity. In the object model their main use is for rapid navigational access to data.

Relationships are abstractions representing associations between objects. In the ODMG standard and many OODBMSs, a relationship is modelled as a pair of relationship signatures, each defining the type of the objects participating in the relationship and the name of a traversal path used to refer to the related objects. The traversal paths are defined in the object types participating in the relationship as inverses of each other (bi-directional relationship). Unidirectional relationships can be defined using single traversal paths. Traversal path cardinality is included in the specification of the target of a traversal path. For example, in the following OODB schema expressed in the ODMG-93 Object Definition Language (ODL) [4]:

```
interface Student
(
    extent students
{
    attribute String name;
    attribute Short student_id;
    relationship Set<Course> takes
        inverse Course::is_taken_by;
}

interface Course
(
)
{
    attribute String name;
    relationship Set<Student> is_taken_by
        inverse Student::takes;
}
```

the traversal paths are *takes* in *Student* and *is_taken_by* in *Course*.

Using traversal paths, OODBMSs are capable of automatically maintaining referential integrity constraints. As the ODMG-93 object model evolves, it is likely that relationships will become first-class types, i.e. they will be named and have their own properties.

The cardinality constraints (1-1, 1-m, m-n) are defined in the traversal path declarations. In the example above, the cardinality ratio between the instances of *Student* and *Course* classes is m-n, and is expressed as a set container. Fixed-cardinality ratios, such as 1-20, can be either expressed as additional business rules associated with traversal paths, or they can be defined using fixed-size arrays. The OODBMSs could then enforce the cardinality constraint on insertion of instances into the relationship.

6.2 Transient object integrity

The traditional database management systems do not manage transient objects, i.e. they do not log changes made to transient objects and therefore cannot undo them. The programming language view, typically adopted by OODBMSs, requires consistency of both transient and persistent objects. In case of transaction aborts, the programmer must ensure that transient object integrity is maintained.

Integrity of transient objects is related to the notion of orthogonality between persistence and type. If persistence and type are orthogonal, then objects of the same type will be allowed to be transient or persistent, and there will be no difference in the way they are used in the program. A distinction is usually made between classes that are persistence-capable and those that are not. Some OODBMSs and ODMG-93 explicitly define persistence-capable classes by deriving them from a base class that provides persistent behaviour. An alternative approach is to treat all classes as persistence-capable.

6.2.1 References between transient and persistent objects

Both transient and persistent objects may participate in relationships. As a general rule, a persistent object cannot refer to a transient object because references to objects stored in different address spaces at different times are meaningless. However, references from transient to persistent objects are always permissible.

ODB developers must decide what to do with persistent objects that refer to transient objects. For example, on writing persistent objects to the database, all references to transient objects could be set to null. On reading them back, it would be the application's responsibility to initialise the references to transient objects with valid addresses.

Alternatively, persistent objects could be stored together with references to transient objects, and, on reading persistent objects back, exceptions would be raised to trigger actions necessary to initialise invalid references.

6.2.2 Transaction consistency for transient objects

As already mentioned, OODBMSs do not typically guarantee the transaction consistency of transient objects. The consistency of transient objects can, however, be guaranteed within a long checkpointed transaction, or between several consecutive transactions.

In general, the states of all transient objects affected by a transaction remain in an application's memory unchanged, even if this transaction aborts. In such cases, it is the application's responsibility to undertake appropriate actions, e.g. undo all changes made to transient objects. It is likely that future OODBMSs will provide some support for undoing transient changes, e.g. by allowing the developer to specify transient objects to be transaction-consistent.

6.2.3 Semantics of object existence

Developers who use OODBs with Smalltalk should pay special attention to the way a particular OODBMS manages transient and persistent objects. In Smalltalk, an object exists as long as it is reachable (referenced within the Smalltalk image). An object is automatically garbage-collected when it is no longer reachable. OODBMSs do not collect and destroy unreferenced persistent objects — persistent objects exist until they are explicitly deleted from the database. In general, Smalltalk should not garbage-collect persistent objects.

One particular problem arises, however, when extents (sets of all instances of a class) are manipulated. Extents are commonly used in object queries. Persistent objects that are only referenced from within their extents for query purposes could qualify for garbage collection.

6.3 Mutability constraints

The ODMG-93 object model distinguishes between mutable (changeable) and immutable (unchangeable) objects. Immutable objects are also referred to as literals. Object mutability is an important semantic constraint that can be used by OODBMSs. If an OODBMS knows that a particular object is immutable, it can prohibit client applications from making any changes to that object. Exclusive locks applied to immutable objects can be ignored. In general, knowledge of mutability can influence the implementation strategy for objects. Current OODBMSs do not provide support for mutability constraints, so developers have to implement their own mechanisms. Mutability constraints are particularly important in the case of co-operative OODB applications.

6.4 Object access constraints

Object- and operation-level access control can be viewed as an additional way of maintaining data consistency in databases. In comparison to relational database management systems, security and access control are an area of weakness for current OODBMSs. Most OODBMSs rely on the underlying operating system for security mechanisms that do not provide sufficient granularity of object access control. As a result, developers very often have to build object access constraints into their applications.

ODBMSs use class specifications as database schema definitions. In OODBMSs supporting C++, the public, private and protected visibility notions could be adopted and used together with the concepts of read and write authorisation. Individual operations could also be protected by execute authorisations. All OODBMSs currently require that attributes used in C++ queries have public visibility.

7. Physical OODB design

This section summarises the main components of the physical OODB design process and gives a number of practical suggestions to improve the overall performance of OODB applications.

7.1 Cache sizes

Disk access is slower than cache access, so caching is an important performance factor — the larger the cache, the bigger can be the working set of objects to which a database application has direct access. It is also essential that the objects used by the application be in the cache when they are needed. Caching is largely under the system's control. OODBMSs normally attempt to predict which objects should be placed in the cache, but in many cases, particularly for interactive applications, a developer's intervention may be very useful.

7.2 Clustering

Clustering means that objects likely to be used together are also stored together on disk. Clustering of objects results in lower disk seek times, a smaller number of pages that need to be moved to the cache, and a lower network overhead. The two basic methods of clustering are programmer-defined clustering and clustering by class. The latter is similar to the table clustering used in relational databases, and is not suitable for applications that need to traverse relationships between objects. Some aspects of clustering are automatically supported by OODBMSs. For example, OODBMSs are responsible for so-called hole-space consolidation in pages with many deleted objects.

7.3 Indexing

Indexing is a well-known technique used to improve performance of value-based queries. There is a clear trade-off between retrieval performance and update overhead that should be considered when using indexes. Normally, developers investigate object access patterns in order to determine which attributes should be indexed; however, some OODBMSs attempt to dynamically construct indexes based on detected access patterns.

Index maintenance overheads can cause considerable problems, for example, when objects to be accessed are very small, when large amounts of data are loaded, or when data integrity constraints, such as uniqueness constraints, are enforced. It is also essential that indexes be placed in their own segments, thereby insuring faster access to index structures.

7.4 Replication

Replication is another technique developers can use to improve the overall performance of OODB applications, the availability of data and site autonomy. Replicas can be stored at the same processing sites (but on different disks) or at different sites (particularly in wide-area networks). Replicated objects can even be stored in the same OODB (for example, clustered by different properties or requirements). Not all objects need the same level of replication. In some cases, developers predetermine replication of objects; in others, it is appropriate for applications or even users to explicitly request that objects be replicated at run time. OODBMSs may determine the appropriate level of replications themselves.

In order to determine the most appropriate policy and techniques for object distribution and replications, the following aspects need to be considered [7]:

- patterns of access to objects, including access to components of aggregate objects,
- relative frequencies of update and query activities,
- tolerance for 'almost-current' data versus the need for continual synchronisation,
- reliability of the environment, with replication providing back-up sources,
- performance requirements and effects of transmission capacities and speeds.

ODBMSs are beginning to offer some features that can assist in object replication, but in most cases it is the responsibility of OODB developers and administrators to manage replication.

7.5 Partitioning

Another aspect of the physical design is partitioning of databases and partitioning of objects:

- objects that happen to be used less frequently than other objects of the same type can be stored in separate containers, thereby making the process of querying the frequently used objects faster, e.g. 'old' objects that are stored in the database for historical reasons could be kept separate from the more recent ones — a mechanism to automatically relocate objects from the current to historical containers could also be implemented,
- objects can be partitioned by classifying their attributes as frequently and seldom used, or as detailed and summary ones — the summary attributes are required when all objects of a particular type need to be used to extract some key information, the detailed attributes are required when greater knowledge of a particular object is needed; in some cases, the number of objects brought to cache can be greatly reduced by splitting an object into two parts and storing the two parts separately.

A number of practical suggestions can be made to improve the overall performance of OODB applications [21]:

- experiment and tune queries to behave more efficiently than those processed by the query optimiser, by, in particular, considering the replacement of declarative queries with procedural ones,
- keep objects as small as possible, as that may result in lower disk seek and network times during access,
- in bulk loading, use a single large transaction rather than several small ones,
- use language-native data types rather than third-party data types from class libraries as these are generic in nature and may introduce too much overhead,
- if a high cache hit is needed for a small set of objects, consider locking them in cache,
- use utilities provided by the vendor for consolidating your fragmented data,
- consider adding direct traversal paths between unconnected objects to improve retrieval or query performance,
- profile applications to determine the times for different application functions.

8. Scalability

Scalability, or the ability to maintain the required performance and functionality as the size of the system grows, is an increasingly important feature of OODBT as it moves into the mainstream of database applications. There is not sufficient evidence of OODB application scalability, mainly because OODBT is a relatively new technology, and the typical OODBs implemented so far are relatively small. A not unlikely scenario is that OODB application developers, when evaluating different OODBMSs, build simple, single-user applications, neglecting the scalability issue altogether.

In general, three types of scalability can be considered in distributed OODB applications:

- scalability in objects (the size of the database),
- scalability in clients (number of users),
- scalability in servers (number of physical databases).

OODBMSs are expected to provide support for the above three types of scalability. The functions of concurrency control, transaction management, schema management, recoverability and access control, as well as support for unique object identifiers that are valid across data stores, should guarantee that single-user database applications scale up to complex distributed database environments with many clients, many servers, and different combinations of local and shared databases.

ODBMSs are migrating to distributed client/server architectures which spread functionality and data across clients and servers to avoid processing bottle-necks. Servers support multithreading, so clusters of objects can be moved simultaneously to many clients. The client-side object managers support indirection to insulate the client application from the database objects stored in the client's cache, thereby maintaining 'physical' integrity of the database. A single logical view of distributed databases allows transparent and dynamic server reconfiguration, so even as server capacity is reached, administrators can simply add new servers and spread objects across them. The result is scalability in both clients and servers.

Several other OODBMS features are directly related to scalability. In addition to the already discussed ability to move persistent objects and allocate them at run time at the same site or at different sites, some OODBMSs provide support for dynamic archiving. This allows selected objects to be moved to off-line storage and make them available again on request. Another feature is support for distributed transactions. OODBMSs supporting this feature should easily scale from a single site to multiple sites.

A somewhat different type of scalability is the ability to support continuously operating OODB applications. A prototype application built and tested in laboratory conditions may not deliver the expected availability. This type of scalability is typically supported by object replication mechanisms (particularly useful in wide-area networks) and by on-line back-up mechanisms with both roll-forward and roll-back capabilities.

Finally, integration scalability should be taken into account. This type of scalability is necessary when OODBT is expected to coexist with other technologies. Some aspects of OODB integration are discussed in section 10.

9. Tool support

The DBMS vendors do not offer just their database engines but also a variety of database tools. The importance of tools cannot be underestimated. An integrated suite of tools makes it easier for developers and users to build, maintain and use database applications. It is estimated that a relational vendor's tools typically account for 25% of its revenue. The availability of database tools, in particular from independent vendors, is a clear sign of software maturity.

In general, database tools can be divided into two groups — database administration tools and application development tools. They typically support the following functions [7]:

- determining where objects are stored, and relocating them as needed to balance resource loads and provide availability,
- declaring who can access which objects, and changing access privileges as needed,
- measuring performance and usage statistics and tuning system resources as required,
- viewing schemas and object instances, and dynamically creating schemas,
- using form-based application generators, report writers, and visual programming tools.

The database management system vendors typically provide database administration tools. Application development tools can come from the database management system vendors or independent vendors. OODB development tools are still rather immature and are mainly supplied by the OODBMS vendors themselves. In the future, however, when OODBMSs are more widely used, OODB development tools will increasingly come from independent vendors. As OODBMSs are usually integrated with object-oriented software development environments, OODB developers expect tools for object modelling, object-oriented programming and GUI building.

The OODBMS vendors are actively developing improved database administration tools. The major problem with current administration tools is that they are not easy to use in distributed computing environments when installing OODBs and configuring OODB applications. An important aspect of these tools is support for monitoring and tuning the performance of OODB applications. Tracking and managing interaction between an OODBMS and external transaction management systems and operating systems is also vital.

The current generation of OODBMSs builds on tight integration with object-oriented programming languages, mainly C++ and Smalltalk. The availability and maturity of programming environments for C++ and Smalltalk, with their class libraries, browsers and debuggers, makes the work of OODB developers a lot more efficient. However, moving OODBT into the mass market requires that OODBMSs also provide interfaces to other, more commonly used languages such as COBOL, SQL, and perhaps Ada and Basic.

An important type of OODB development tool is an object-modelling tool. Object modelling tools are usually regarded as part of an object-oriented programming environment. They typically provide support for a variety of object-oriented analysis and design methods. Modelling tools can be used as OODB schema facilities that enable developers to design persistent class libraries and automatically generate parts of the code. There is, however, still a need for object-modelling tools that are specifically tailored to OODB requirements (see section 2).

A frequent criticism raised against OODBMSs is that they usually do not offer high-level development tools, such as 4GLs, form generators and report writers. The problem obviously lies in the definition of a high-level OODB language that is expressive enough in object-oriented terms. Some suggestions have been made that Smalltalk could be marketed as a 4GL for OODBs. Smalltalk has been accepted by some information system developers as an effective prototyping tool for application development. It is argued that, in this capacity, Smalltalk may replace the current generation of 4GLs.

Another major challenge for OODBMS vendors is to provide a consistent framework for tighter integration of all OODB tools. All too often, OODB tools are built as stand-alone packages. The transition between the different stages of OODB development process (analysis, design, installation, configuration and use) must be made much smoother.

The OODBMS vendors justifiably concentrate on improving their OODB engines and developing OODB administrator tools. The increasing popularity of OODBT has recently attracted independent vendors who have

identified a market niche for OODB development tools, such as object 4GLs and object report writers. Typical OODB tools provide support for graphical representation of objects, collections and relationships, as well as navigation through large sets of data and direct modification of its content. Other development tools include GUI builders, data dictionaries, query definition tools and report writers.

10. Integration with other technologies

The advantages of OODBT are well known and generally acknowledged. So far, however, OODBT has mainly been used to build highly specialised data-processing applications. Such applications are typically developed from scratch, or built as extensions to existing systems that already use an object-oriented paradigm. In commercial environments, it is unrealistic to expect that organisations would be willing, in the short term at least, to replace their existing, proven database systems with OODBs, no matter how promising the new technology is. Assuming that OODBT is the next-generation database technology, a far more practical approach in the long run is to look for evolutionary means to introduce and integrate the new technology with the old ones.

In reality, there is a genuine need for such integration in distributed information systems. The motivation for integration might be a long-term goal of switching to a superior technology, achieving better performance, extending system functionality, increasing programmers' productivity, or using more expressive information modelling techniques. They are all valid considerations, essential from the perspective of users of large-scale database applications. The OODBT vendors are responding to these requirements by developing integration tools. Some experts believe that it is the only way for OODBT to enter the mainstream of database applications.

There are several ways in which OODBT can be integrated with other database technologies. One of them uses distributed object technology to create a software infrastructure for making database systems, including OODB systems, interoperable. This approach has not been explored sufficiently yet, and there are no general-purpose tools supporting distributed-object integration. Consequently, information system integrators using distributed object technology have to come up with their own, problem-tailored solutions [22]. Several OODBMS vendors have announced plans to include some distributed-object elements, such as the Object Management Group's common object transaction service [23], in their products. The combined use of DOT and OODBT is discussed in greater detail in Machura and Jebb [24].

Another approach to integration, increasingly promoted by the OODBMS vendors, is based on so-called mapping technologies that provide connectivity between OODBs and

legacy systems, mostly those using relational database technology [25]. The mapping technologies support OODBC foreign access to OODBMSs and native OODBMS access to relational database management systems (the latter is referred to as gateway technology). OODBC is a Microsoft-supported subset of the SQL Access Group server protocol. Several connectivity products from OODBMS vendors are already available on the market.

Both gateways and OODBC clients must provide a way of synchronising transactions distributed across multiple databases. XA is a standard that defines a distributed two-phase commit protocol to which many relational database management systems already adhere. OODB products are also expected to provide XA compliance.

11. Conclusions

Architecturally, OODBT is the ideal basis for systems that require the ability to handle complex database structures such as composite objects, and to work across multiple servers, multiple databases, multiple schemas, and multiple interfaces.

In general, performance requirements should be considered relatively early in the design process of OODB applications. The key issues to be addressed are object locking, location of operation execution, object integrity maintenance and security, physical database design (cache sizes, clustering, indexing, replication, partitioning), scalability, tool support and integration with other technologies.

As to high-level design, the existing object-oriented analysis methods provide adequate support for conceptual (and logical) design of OODBs. The object-oriented design methods, however, do not meet the requirements of OODB systems. For example, distribution and transaction processing are regarded as low-level design issues and therefore are not supported at all. As a consequence, designers of distributed object systems too often carry out their work in an *ad hoc* manner.

In addition to producing distributed object models, high-level design should also deal with issues such as scalability requirements, expected transaction processing patterns and overall security of the distributed system. Object-oriented design methods will have to be enhanced to meet the new requirements of physical design of OODBs. In general, they should support concurrent access to shared objects (transaction strategy, clustering strategy, user access, changing class definitions and class associations) and efficient search mechanisms (indexes).

References

- 1 Atkinson M, Bancilhon F, DeWitt D, Kittrich K, Maier D and Zdonik S: 'The object-oriented database system manifesto', Proceedings of the International Conference on Deductive and Object-Oriented Databases (DOOD '89), pp 40—57, Kyoto, Japan (1989).
- 2 'Third-generation database system manifesto', The Committee for Advanced DBMS Function, Memorandum UCB/ERL M90/20, University of California, Berkeley (1990).
- 3 Darwen H and Date C: 'The third manifesto', SIGMOD RECORD, 24, No 1 (March 1995).
- 4 Cattell R (Ed): 'The object database standard: ODMG-93 (release 1.1)', Morgan Kaufmann, <http://www.odmg.org> (1994).
- 5 Melton J (Ed): 'Database language SQL3', ISO/ANSI Working Draft (1994).
- 6 Two contrasting opinions on the ODMG-93, Object Magazine, pp 65—76 (September 1994).
- 7 Loomis M: 'Object databases: the essentials', Addison-Wesley (1995).
- 8 Cattell R: 'Object data management: object-oriented and extended relational database systems', (2nd edition), Addison-Wesley (1995).
- 9 Rao B: 'Object-oriented databases', McGraw-Hill (1994).
- 10 Kemper A and Moerkotte G: 'Object-oriented database management', Applications in Engineering and Computer Science, Prentice-Hall (1994).
- 11 Kotes-Dittrich A and Dittrich K: 'Where object-oriented DBMSs should do better: a critique based on early experiences', in Kim W (Ed): 'Modern database systems: the object model, interoperability and beyond', Addison-Wesley (1995).
- 12 Kim W: 'Object-oriented database systems: promises, reality, and future', in Kim W (Ed): 'Modern database systems: the object model, interoperability and beyond', Addison-Wesley (1995).
- 13 'DBMS needs assessment for objects', version 3.0, Barry & Associates, 14041 Burnhaven Dr, Suite 103, Burnsville, MN 55337 (1995).
- 14 Manola F: 'An evaluation of object-oriented DBMS developments', GTE Laboratory Technical Report (1994).
- 15 Barry D: 'OODBMS benchmarks', Object Magazine, pp 24—26 (October 1994).
- 16 Chaudri A: 'An annotated bibliography of benchmarks for object databases', SIGMOD RECORD, 24, No 1 (March 1995).
- 17 Jacobson I, Christerson M, Jonsson P and Overgaard G: 'Object-oriented software engineering — a use case driven approach', Addison-Wesley (1992).
- 18 Rumbaugh J, Blaha M, Premerlani W, Eddy F and Lorensen W: 'Object-oriented modelling and design', Prentice-Hall (1991).
- 19 'Unified modelling language, version 1.0', Object Management Group (1997).
- 20 Barry D: 'The importance of understanding OODBMS features', Object Magazine, pp 76—78 (May 1994).
- 21 Gorur A and Fotino J: 'The physical design of object databases', Object Magazine, pp 51—53 (July/August 1995).
- 22 San Soucie M and Almarode J: 'Supporting enterprise-wide business objects', Object Magazine, pp 44—59 (September 1995).
- 23 'Object Transaction Service', Object Management Group, <http://www.omg.org/docs/1994/94-08-04.ps>, (1994).
- 24 Machura M and Jebb T: 'Distributed objects and object databases', BT Technol J, 15, No 4, pp 207—219 (October 1997).
- 25 Talbot S: 'Connectivity: gateways to success', Object Expert, 1, No 3, pp 43—46 (1996).



Marek Machura graduated from Technical University of Warsaw with an MSc in Control Engineering and Computing, and received a PhD in Computing from the same university in 1974. He worked as a researcher in the Institute of Atomic Energy, Warsaw, Stuttgart University, and Engineering Design Research Centre, Glasgow. He has conducted research and led a number of R&D projects in the areas of numerical software, intelligent front-ends to evolving software systems, and design support environments. Currently he is a senior lecturer at Bournemouth University. His main research interests include distributed object computing and object databases. Currently he is investigating new approaches to distributed data management in self-organising information systems.



Tim Jebb joined BT in 1984. He initially worked in a group providing technical support for IBM mainframes, specialising in data communications. In 1986 he moved into a technical support role for the BT WatchMan network management product. From there he joined BT Laboratories, developing techniques for interworking between computers using proprietary protocols. He has recently been working on distributed information management, and information services, contributing to the BT Touchpoint service. He is currently developing Oracle software for transferring information between an Oracle system and an ObjectStore system.